Behavioral Ontology Correctness: From Syntax Checking to Evidence-Valued Verification via SUMO → GF → OSLF → World Models

(AIrxiv Working Paper, 2026-02-20)

Abstract

Large formal ontologies like the Suggested Upper Merged Ontology (SUMO) contain tens of thousands of axioms in the Knowledge Interchange Format (KIF). Despite decades of automated-reasoning work, known bug classes persist: arity mismatches, type confusion, missing arguments, and—more subtly—axioms whose *operational consequences* diverge from their intended meaning. Existing debugging methods (glass-box reasoner diagnostics, white-box test generation, black-box competency questions, OntoClean metaproperty analysis, design-pattern templates) each address a layer of correctness but remain disconnected and leave behavioral adequacy unexamined.

We propose a four-stage pipeline—SUMO → SUMO-GF → GF → OSLF → WM—that provides *behavioral* and *evidential* layers of ontology verification currently absent from the literature. Stage 1 maps KIF relations and classes to Grammatical Framework (GF) abstract syntax, where the grammar's type discipline catches arity and sort errors at parse time. Stage 2 synthesizes Operational Semantics in Logical Form (OSLF) from the grammar, automatically deriving modal behavioral types ($\Diamond$, $\Box$) and a Galois connection. Stage 3 interprets behavioral types as evidence-valued world-model queries, grounding ontological assertions in PLN Evidence counts $\langle n^+, n^- \rangle$ and flagging ungroundable claims.

The GF → OSLF → WM core of this pipeline is implemented in Lean 4 in the referenced Mettapedia pipeline slice (not the entire repository), with no `sorry`/axiom placeholders in that slice. We further propose a *convergent formalization-driven repair loop* that uses the pipeline's type, behavioral, and evidential signals to disambiguate and verify ontology corrections—going beyond detection to principled repair. We survey the ontology debugging literature, document SUMO's known issues, and argue that, in the scoped setting of this draft, the proposed pipeline can integrate major strengths of prior approaches while adding behavioral-correctness and repair layers.

Status note. Implementation-status and count claims in this draft are snapshot-scoped; verify against current repository trackers and build outputs before citing as current state. Lean-proven claims are only those tied to explicit Lean file/theorem references; broader mathematical or historical claims are literature-backed context and not asserted as Lean-proven unless explicitly stated.

1 Introduction

The Suggested Upper Merged Ontology (SUMO) is one of the largest freely available formal ontologies, comprising over 20 000 axioms in the Knowledge Interchange Format (KIF) across 113 domain files [1]. It has been the subject of sustained automated-theorem-proving (ATP) effort, including annual prizes at the CADE ATP System Competition (CASC) [2, 3]. Despite this, known bugs persist:

arity mismatches between relation declarations and their usage sites, type confusion (e.g., applying `contraryAttribute` to a `Process` where an `Attribute` is expected), missing arguments in domain-module facts, and semantically problematic axioms that pass syntactic checks but produce unintended inferences.

The ontology engineering community has developed several debugging paradigms:

- **Glass-box**: Tableau-internal axiom pinpointing for unsatisfiable OWL classes [6].
- **White-box**: Automatic test generation from axiom syntax, ATP-based redundancy detection [7].
- **Black-box**: WordNet-based competency questions for external validation [8].
- **OntoClean**: Philosophical metaproperty analysis (rigidity, identity, unity) for taxonomic adequacy [9].
- **Design patterns**: Modular ontology templates for prevention [10].

Each addresses a genuine layer of correctness, but they remain disconnected: no framework integrates syntactic well-formedness, semantic consistency, *behavioral adequacy* (do the axioms' operational consequences match the intended domain dynamics?), and *evidential grounding* (can the asserted knowledge be supported by observation?).

Thesis. We argue that the missing layers are *behavioral* and *evidential* correctness, and that operational semantics provides the bridge. Concretely, we propose the pipeline

$$\text{SUMO} \xrightarrow{\text{Stage 1}} \text{SUMO-GF} \xrightarrow{\text{Stage 2}} \text{OSLF} \xrightarrow{\text{Stage 3}} \text{WM}$$

where:

1. **Stage 1** (SUMO → SUMO-GF): translates KIF into Grammatical Framework abstract syntax, gaining grammar-level sort and arity checking.
2. **Stage 2** (GF → OSLF): synthesizes behavioral types ($\Diamond$, $\Box$, native types) from operational semantics.
3. **Stage 3** (OSLF → WM): interprets behavioral properties as evidence-valued world-model queries.

The GF → OSLF → WM core is implemented in the referenced Mettapedia Lean 4 pipeline slice with no `sorry`/axiom placeholders in that slice (hundreds of generated grammar rules, proven Galois connection, and proven evidence-valued semantics in the cited modules). The SUMO → GF stage builds on our existing `sumo-lean4` KIF checker and HOL emitter, and on Ranta et al.'s typeful ontology verbalization work [12].

Contributions.

1. A four-level correctness taxonomy for ontologies (§2).
2. A survey of SUMO's structure, known bugs, and their root causes (§3).
3. A systematic review of ontology debugging methods and what each misses (§4).
4. The SUMO → GF → OSLF → WM pipeline architecture (§5).
5. A comparison with direct SUMO → Lean approaches, showing the pipeline's advantages (§8).
6. A worked example of a real SUMO bug flowing through the pipeline (§9).
7. A formalization-driven repair methodology with canary theorems and behavioral disambiguation (§6).

2 What Makes an Ontology Correct?

We identify four levels of ontology correctness, each subsuming the previous:

Definition 2.1 (Syntactic well-formedness)**.** Every axiom conforms to the language grammar: balanced parentheses, declared relation names, correct arity at each usage site, and well-sorted arguments matching `domain`/`range` declarations.

Definition 2.2 (Semantic consistency)**.** The axiom set has at least one model (is satisfiable). No subset of axioms entails $\perp$. Equivalently: no class is forced to be empty by the axioms unless intended (unsatisfiable-class debugging in the sense of [6]).

Definition 2.3 (Taxonomic adequacy)**.** The subsumption hierarchy respects philosophical metaproperties:

- **Rigidity**: a rigid property cannot cease to hold for an individual (e.g., *Human* is rigid; *Student* is anti-rigid).
- **Identity**: a sortal carries an identity criterion that its subsorts inherit.
- **Unity**: instances of a unitary property share the same "whole-part" structure.

This is the domain of OntoClean [9]. A taxonomy is *adequate* if no anti-rigid class subsumes a rigid one, no class carries conflicting identity criteria, etc.

Definition 2.4 (Behavioral correctness)**.** The operational consequences of the axioms match the intended domain dynamics. Formally: given an operational semantics $\Rightarrow$ derived from the ontology's axioms, for every intended domain scenario σ:

1. If the domain intends "ϕ is possible from state s," then $s \models \Diamond\, \phi$ (the operational semantics produces a reduction path to a state satisfying ϕ).
2. If the domain intends "ϕ is necessary for state s," then $s \models \Box\, \phi$ (every predecessor of s satisfies ϕ).
3. No axiom produces zero evidence under world-model evaluation unless it is explicitly a vacuous assertion.

The first two levels are well-served by existing tools (ATP for consistency, KIF checkers for syntax). Level 3 (OntoClean) is mature but manual and requires philosophical expertise. In the source set surveyed for this draft, we did not find an existing ontology-engineering methodology that addresses Level 4 behavioral correctness as an end-to-end verification layer. Table 1 summarizes the landscape.

Table 1: Ontology correctness levels and which methods address them.

	KIF checker	*ATP (E/Vampire)*	*Glass-box*	*OntoClean*	*Keet testing*	*Axiom weak.*	**Pipeline**
Syntactic well-formedness	✓				✓		✓
Semantic consistency		✓	✓		(✓)	✓	✓
Taxonomic adequacy				✓			(✓)
Behavioral correctness							✓
Repair guidance			(✓)			✓*	✓

* $\mathcal{EL}$ only (Troquard et al. 2018, Baader & Kriegel 2023).

Remark 2.1. Level 4 (behavioral correctness) presupposes an operational reading of the ontology. This is natural for ontologies that describe processes, events, causation, and change—a large fraction

of any upper ontology including SUMO. For purely classificatory ontologies (taxonomies without process axioms), levels 1–3 may suffice.

3 SUMO: Structure and Known Issues

3.1 KIF Syntax and Semantics

The Knowledge Interchange Format [5] uses a LISP-compatible S-expression syntax. Key features distinguishing KIF from standard first-order logic:

- **Free syntax**: no syntactic distinction between object and predicate variables. A variable `?R` can appear in predicate position: `(?R ?X ?Y)`.
- **Variadic relations**: a relation need not have a unique arity. `@ROW` variables allow variable-length argument lists.
- **Row variables**: `@ROW` splices a sequence of terms into an argument list, taking the language beyond first-order.
- **Self-application**: a relation can appear as an argument to itself or to `instance`/`subclass`.

Hayes and Menzel [5] gave a formal semantics for a cleaned-up variant called SKIF, noting that the full use of row variables takes the language beyond first-order expressiveness.

3.2 SUMO Architecture

SUMO is organized into:

- **Merge.kif** (607 KB): the upper ontology—classes like `Entity`, `Physical`, `Process`, `Attribute`, and foundational relations (`instance`, `subclass`, `domain`, `range`).
- **Mid-level-ontology.kif** (1.1 MB): intermediate concepts (countries, organizations, artifacts, biological processes).
- **Domain modules** (111 files): specialized knowledge (Cars, Geography, Economy, Languages, etc.).
- **domainEnglishFormat.kif** (3.7 MB): English natural language generation templates for every relation.

Total: 113 KIF files, approximately 1.6 GB including natural-language annotations.

3.3 Known Bug Classes

We have encountered or documented the following bug classes through our `sumo-lean4` checker and manual inspection[1]:

1. Arity mismatches. The predicates `classmate` and `colleague` are explicitly ternary (person, person, context), but were also declared as instances of `IrreflexiveRelation`—a class defined in `Merge.kif` to apply only to binary relations. Our checker flagged this; the fix was to remove the `IrreflexiveRelation` instances and add explicit ternary irreflexivity axioms:

```
(forall (?A ?COURSE)
  (not (classmate ?A ?A ?COURSE)))
```

2. Missing arguments. In `mondial.kif`, four Factbook-style demographics facts were missing the region argument: `beliefGroupPercentInRegion` and `ethnicityPercentInRegion` had two arguments instead of three. The checker normalized them to the correct three-argument form.

[1] See also `PossibleBugsInSUMO_HopefullySolvedByNowBySomeone.kif` in our repository.

3. Argument normalization. The predicate `depth` has a three-argument signature (object, reference-point, measure), but `mondial.kif` contained hundreds of two-argument facts `(depth Seas_X 1234)`. These were normalized to `(depth Seas_X SeaLevel (MeasureFn 1234 Meter))`.

4. Type confusion. In `emotion.kif`, `(contraryAttribute Pleasure Pain)` treats `Pain` as an `Attribute`, but SUMO's own taxonomy classifies `Pain` as a `PathologicalProcess` (a subclass of `Process`, not `Attribute`). This is a genuine ontological error: the axiom is well-formed in KIF's free syntax but violates the intended sort discipline.

5. Semantic ambiguity. The predicate `monetaryValue` is declared with `(domain monetaryValue 1 Physical)`, but the axioms defining `WealthFn` use it for abstract financial quantities (`PropertyFn`) that may not be `Physical` in the intended sense.

3.4 ATP Experience

Pease and Sutcliffe [3] reported sustained efforts to make SUMO amenable to ATP systems at CASC. The main challenges were:

- Row variable elimination (requires higher-order treatment or bounded unfolding).
- Self-referential axioms (SUMO asserts things about its own predicates).
- Scale: first-order provers struggle with the full 20k-axiom set; modular decomposition is essential.

Benzmüller [13] later showed how to faithfully embed various logics (including SUMO's modal and higher-order fragments) into classical HOL, providing both "deep" (syntax-preserving) and "shallow" (semantics-preserving) embeddings. This is closer to our approach but still treats axioms as opaque assertions rather than extracting operational behavior.

4 The Ontology Debugging Literature

4.1 Glass-Box Debugging (Kalyanpur et al. 2005)

Kalyanpur et al. [6] introduced glass-box debugging for OWL ontologies, using the internals of description-logic tableaux reasoners (specifically Pellet) to:

- Distinguish *root* unsatisfiable classes from *derived* ones (those that are unsatisfiable only because a root class is).
- Extract *justifications*: minimal subsets of axioms responsible for an unsatisfiability.
- Present structured explanations to users in the Swoop ontology editor.

Strength: Pinpoints the *root cause* of logical errors, not just their symptoms.

Limitation: Restricted to description-logic (OWL) ontologies. SUMO's full KIF (with row variables, self-application, and unrestricted quantification) is far beyond OWL's expressiveness. The tableaux-based approach does not extend to full FOL.

4.2 White-Box Testing (Keet & Ławrynowicz 2017)

Keet and Ławrynowicz [7] proposed automatic test generation from the syntactic structure of FOL axioms:

- Each axiom pattern (universal, existential, disjunction, etc.) generates a family of positive and negative test cases.

- Tests are run against an ATP to check whether the ontology entails (or correctly rejects) each test case.
- Redundant axioms are flagged when removing one does not change the test outcomes.

Strength: Fully automatic, works with FOL.

Limitation: FOL entailment is semi-decidable; the ATP may time out on hard test cases. More fundamentally, the tests check *what the ontology entails*, not *whether those entailments match the intended domain behavior*.

4.3 Black-Box Testing (Keet & Ławrynowicz 2017)

The same authors proposed black-box testing using WordNet [8] as an external oracle:

- WordNet's synsets and hypernymy provide expected subsumption relationships.
- The ontology is queried: does it entail that A subsumes B when WordNet says it should?
- Discrepancies identify potential errors or modeling choices.

Strength: External validation—not circular.

Limitation: Depends entirely on the quality of the ontology-to-WordNet mapping. WordNet's coverage is uneven across domains, and mapping ambiguity introduces both false positives and false negatives.

4.4 OntoClean (Guarino & Welty)

OntoClean [9] is a philosophical methodology for evaluating taxonomic structures. Each class is tagged with metaproperties:

- **Rigidity (+R/-R/∼R)**: does the property hold necessarily for its instances?
- **Identity (+I/-I)**: does the class carry or inherit an identity criterion?
- **Unity (+U/-U/∼U)**: do instances share a unifying whole-part relation?
- **Dependence (+D/-D)**: does the class's existence depend on another entity?

Constraint rules (e.g., "an anti-rigid class must not subsume a rigid one") identify taxonomic errors.

Strength: Deep philosophical analysis of "is this hierarchy right?"—catching errors that no amount of logical consistency checking can find.

Limitation: Entirely manual. The metaproperties must be assigned by a human expert, and there is no tool support for propagation or conflict detection at scale. We argue below (§5.6) that the pipeline's evidence-valued semantics can *automate* rigidity assessment, replacing the binary $\{0, 1\}$ tags with graded evidence from corpus evaluation.

4.5 Ontology Design Patterns (Gangemi & Presutti)

Gangemi and Presutti [10] advocated reusable ontology design patterns—modular templates for common modeling situations (parthood, participation, classification, etc.).

Strength: Prevents errors by providing vetted templates.

Limitation: Post-hoc debugging is still needed for axioms that do not fit a known pattern, and patterns do not address behavioral correctness.

4.6 Summary: A Layered but Disconnected Landscape

No existing framework integrates all four correctness levels from §2. Glass-box and white-box methods address levels 1–2 (syntax, consistency). OntoClean addresses level 3 (adequacy). *Level 4 (behavioral correctness) is unaddressed.*

Moreover, even where bugs are detected, the existing literature provides limited guidance on *repair*. Axiom weakening [19] and optimal repair [20] offer principled repair operators, but their theory is developed only for the lightweight description logic $\mathcal{EL}$—far less expressive than SUMO's full KIF. Adimen-SUMO [22] applied ATP-based diagnosis to re-engineer 88% of SUMO, but without behavioral or evidential signals to disambiguate among repair candidates.

The pipeline we propose (§5) provides levels 1 and 4, and inherits level 2 from the underlying ATP and type system. For level 3, the pipeline's evidence semantics can *partially automate* OntoClean's manual metaproperty assignment (§5.6). Crucially, it also provides a *repair* methodology (§6) that uses the pipeline's multi-level diagnostics to guide principled corrections.

5 The Pipeline: SUMO → GF → OSLF → WM

5.1 Overview

SUMO	$\xrightarrow{\text{KIF parse}}$	SUMO-GF	$\xrightarrow{\text{OSLF synthesis}}$	OSLF	$\xrightarrow{\text{evidence eval}}$	WM
axioms		GF abstract syntax		$\Diamond$, $\Box$, native types		$\langle n^+, n^- \rangle$

Each stage catches a different class of errors:

Stage	**Bug class caught**	**Mechanism**
1. SUMO → GF	Arity, sort errors	Grammar type discipline
2. GF → OSLF	Behavioral inconsistencies	$\Diamond/\Box$ violation
3. OSLF → WM	Ungroundable assertions	Zero-evidence detection
	Scope ambiguities	Evidence ordering

5.2 Stage 1: SUMO → SUMO-GF

The Grammatical Framework (GF) [11] is a grammar formalism with a type-theoretic abstract syntax and language-specific concrete syntaxes (linearizations). The key insight for ontology debugging is that GF's abstract syntax is a *typed* term language: every function signature declares the sorts of its arguments and return value.

Mapping. SUMO classes become GF *categories*, and SUMO relations become GF *function signatures*:

SUMO (KIF)	GF Abstract Syntax
`(subclass Dog Animal)`	`cat Dog ; cat Animal`
`(domain bites 1 Animal)`	`fun bites : Animal -> Entity -> Prop ;`
`(domain bites 2 Entity)`	
`(instance Fido Dog)`	`fun Fido : Dog ;`

`domain` and `range` declarations in SUMO provide the sort information; our existing `sumo-lean4` checker already extracts these. The translation is structurally identical to the HOL emitter in `sumo-lean4/SumoKif/HolEmit.lean`, but targets GF categories instead of Lean types.

What this catches. An axiom like `(contraryAttribute Pleasure Pain)` requires `Pain` to be of category `Attribute`. SUMO's own taxonomy declares `Pain` as a subclass of `PathologicProcess`—a `Process`, not an `Attribute`. If the encoding faithfully follows SUMO's declarations, the GF type checker rejects the abstract tree—*before any reasoning occurs*.

Interestingly, the original SUMO-GF grammar [23] silently reclassifies `Pain` as `Ind EmotionalState` (an `Attribute`), making the axiom type-correct. This is an ontologically defensible choice—Pain is a sensation *caused by* a pathological process, not the process itself—but it represents an undocumented repair. Our pipeline (Appendix B) instead follows SUMO's own declarations faithfully, detects the conflict, and logs the repair decision with evidence and canary theorems (§B.5).

Multilingual verbalization. As a bonus, GF's concrete syntax mechanism enables automatic natural-language verbalization of SUMO axioms, following Ranta et al. [12]. An axiom that parses correctly in the grammar can be linearized to English, Czech, or any other language with a concrete syntax—providing a human-readable sanity check.

5.3 Stage 2: GF → OSLF

Operational Semantics in Logical Form (OSLF) takes a grammar (presented as a `LanguageDef`: sorts, constructors, equations, rewrite rules) and automatically synthesizes a behavioral type system.

LanguageDef. Our Mettapedia library represents a language as:

- **Sorts**: the categories (from GF).
- **Terms** (`GrammarRule`): labeled constructors with typed parameters and syntax patterns.
- **Equations**: structural equality/normalization rules.
- **Rewrite rules**: one-step operational semantics.
- **Premises**: side conditions (freshness, congruence, relation queries).

The existing GF → OSLF bridge in Mettapedia (`Languages/GF/OSLFBridge.lean`) translates 170 GF RGL function signatures into 985 grammar rules, and our pipeline `langOSLF` synthesizes:

- A reduction relation $\Rightarrow$ from the rewrite rules.
- A possibility operator $\Diamond\,\phi$: "a state satisfying ϕ is reachable."
- A necessity operator $\Box\,\phi$: "all predecessors satisfy ϕ."
- A Galois connection $\Diamond \dashv \Box$ (proven automatically from a generic theorem).
- Native types as (sort, predicate) pairs via the Grothendieck construction.

What this catches. Suppose SUMO asserts "every Human has a parent" via:

```
(=> (instance ?H Human)
    (exists (?P) (parent ?H ?P)))
```

In the operational semantics derived from SUMO's axioms, if there is no rewrite rule that produces a `parent` relation from a `Human` state, then $s \not\models \Diamond(\texttt{parent}(h, p))$ for any state s containing a Human h. This is a *behavioral* bug: the ontology asserts an existential that its own operational semantics cannot witness.

5.4 Stage 3: OSLF → WM

The final stage interprets OSLF behavioral types as evidence-valued world-model queries.

Evidence. Following the PLN evidence quantale [15], evidence is a pair $\langle n^+, n^- \rangle \in \mathbb{R}^\infty_{\geq 0} \times \mathbb{R}^\infty_{\geq 0}$, where n^+ counts positive evidence and n^- counts negative evidence. The strength (point estimate) is $s = n^+/(n^+ + n^-)$. Evidence forms a complete lattice and a commutative quantale under coordinatewise multiplication.

World-model evaluation. Given a quantified formula ϕ (produced by the GF $\to$ Store $\to$ QFormula pipeline), the world-model evaluator qsemE2 produces:

$$\text{qsemE2}(\phi) \in \text{Ev}$$

where:

- Atomic formulas are evaluated by querying an evidence-atom semantics $I : \text{Pred} \times \text{Args} \times \text{Point} \to \text{Ev}$.
- $\forall$ is interpreted as $\bigwedge$ (infimum over domain).
- $\exists$ is interpreted as $\bigvee$ (supremum over domain).
- $\Diamond$ and $\Box$ are interpreted via reduction reachability.

What this catches.

- **Ungroundable assertions**: if $\text{qsemE2}(\phi) = \langle 0, 0 \rangle$ (zero evidence), the ontology asserts something it cannot ground in any evidence source.
- **Contradictory evidence**: if $\text{qsemE2}(\phi \wedge \psi) \ll \text{qsemE2}(\phi) \otimes \text{qsemE2}(\psi)$ (evidence for the conjunction is much less than the product of evidence for the conjuncts), the axioms are in tension.
- **Scope ambiguities**: for quantified SUMO axioms, the inverse-scope reading may produce different evidence than the surface reading. Our `scope_ordering_NT` theorem in Mettapedia proves that inverse $\leq$ surface in the evidence lattice, so a significant gap flags a scope-sensitive assertion.

5.5 Compositionality

A key advantage of the pipeline is *compositionality*. GF's abstract syntax is Frege-compositional by construction: the meaning of a complex expression is determined by the meanings of its parts and their mode of combination. OSLF's type synthesis preserves this: the behavioral type of a compound term is determined by the behavioral types of its subterms. The world-model evaluator is a compositional denotational semantics.

This means that if two axioms are individually well-typed and behaviorally correct, their combination is type-correct under the compositional typing theorem used in this pipeline slice. *Compositional bugs*—where two individually valid axioms interact to produce an ill-typed behavior—are caught at the OSLF level by the native type discipline, specifically by the Grothendieck construction over the subobject fibration (`CategoryTheory/NativeTypeTheory.lean`).

5.6 Automating OntoClean via Evidence-Valued Rigidity

OntoClean's central metaproperty, *rigidity*, asks: "if x is a C, must x *necessarily* be a C?" Traditional OntoClean assigns a binary tag: +R (rigid), −R (non-rigid), or ∼R (anti-rigid). The assignment is manual and requires philosophical expertise.

We observe that the pipeline's evidence-valued semantics provides a natural *automation* of rigidity assessment, replacing the binary $\{0, 1\}$ tag with a graded measure in $[0, 1]$.

Formalization. For each ontology class C, define the *rigidity evidence* as the world-model evaluation of the claim that some instance of C could cease to be a C:

$$\text{rig-ev}(C) \;:=\; \text{qsemE2}\big(\exists x.\ \text{instanceOf}(x, C) \;\wedge\; \Diamond(\neg\,\text{instanceOf}(x, C))\big)$$

Here $\Diamond$ is the pipeline's behavioral possibility operator, which checks whether $\neg\,\text{instanceOf}(x, C)$ is reachable under the ontology's rewrite dynamics. The result is an evidence pair $\langle n^+, n^- \rangle$.

The *graded rigidity* of C is then:

$$\rho(C) \;:=\; 1 - s(\text{rig-ev}(C)) \;=\; 1 - \frac{n^+}{n^+ + n^-}$$

where s is the PLN strength. When no evidence is found for instances losing class membership ($n^+ = 0$), $\rho(C) = 1$ (fully rigid). When abundant evidence exists ($n^+ \gg n^-$), $\rho(C) \approx 0$ (anti-rigid).

Examples. Consider three SUMO classes evaluated against a commonsense corpus:

Class	**Evidence for change**	$\boldsymbol{\rho(C)}$	**OntoClean tag**
`Human`	No factual examples of becoming non-Human	≈ 1.0	+R
`Student`	Graduation, dropout ($n^+ \gg 0$)	≈ 0.1	∼R
`Physical`	No evidence of physical→abstract	$= 1.0$	+R
`President`	Term endings, impeachment ($n^+ > 0$)	≈ 0.2	∼R

Domain-relative rigidity. A key advantage of the evidence-based approach is that rigidity becomes *domain-relative*: the same class can have different rigidity values depending on the world model's corpus. A medical ontology evaluated against clinical records will assign $\rho(\texttt{Human}) \approx 1.0$, while a science-fiction ontology might assign a lower value. This is not a defect but a feature: rigidity genuinely *should* depend on the domain of discourse. Traditional OntoClean obscures this by forcing a single universal tag.

Constraint propagation. OntoClean's constraint rules translate directly to evidence inequalities. The rule "an anti-rigid class must not subsume a rigid one" becomes: if $C_1 \sqsubseteq C_2$ and $\rho(C_2) > \rho(C_1)$, then the subsumption is suspect—a child class is more changeable than its parent, violating monotonicity of rigidity along the taxonomy. These inequalities can be checked *automatically* across the entire ontology once graded rigidity values are computed.

Relation to other metaproperties. The same evidence-valued approach extends to OntoClean's other metaproperties. *Identity* can be assessed by evaluating evidence for identity criteria: $\text{qsemE2}(\exists f.\ \forall x, y : C.\ f(x) = f(y) \to x = y)$. *Unity* can be assessed via evidence for part-whole coherence. We leave the formal development of these extensions to future work, noting that the pipeline's evidence quantale provides the necessary algebraic structure in each case.

6 From Detection to Repair: The Formalization-Driven Loop

The pipeline of §5 *detects* bugs across four correctness levels. But detection is only half the story: a principled *repair* methodology is needed to close the loop. This section surveys existing ontology repair approaches, identifies their limitations for full-FOL ontologies like SUMO, and proposes a convergent formalization-driven repair loop that exploits the pipeline's behavioral and evidential signals.

6.1 The Ontology Repair Literature

Justification-based repair. Kalyanpur et al. [6] introduced *Minimal Unsatisfiability-Preserving Sub-TBoxes* (MUPS): the smallest subset of axioms sufficient for a concept to be unsatisfiable. Horridge et al. [25] refined this to *laconic justifications* that eliminate redundancy within justifications. Repair proceeds by removing or modifying at least one axiom from each justification.

Limitation: These methods require a complete reasoner (typically a DL tableaux); they do not extend to SUMO's full KIF with row variables and self-application. Moreover, deletion-based repair loses information.

Axiom weakening. Troquard et al. [19] proposed *weakening* axioms rather than deleting them, using refinement operators from inductive logic programming. For example, $A \sqsubseteq B \sqcap C$ may be weakened to $A \sqsubseteq B$ (dropping a conjunct) rather than deleted entirely. This preserves significantly more of the original knowledge.

Baader and Kriegel [20, 21] showed that optimal repair in $\mathcal{EL}$ TBoxes corresponds precisely to *pseudo-contraction* in AGM belief change theory: the repair that removes the fewest entailments while restoring consistency. They also introduced *fixed-premise repairs*, where some axioms are trusted ("fixed") and only the remainder is modified.

Limitation: The theory is developed only for $\mathcal{EL}$, with recent extensions to Horn-DLs [21]. SUMO's full first-order expressiveness (row variables, unrestricted quantification, self-reference) is far beyond these fragments.

Adimen-SUMO. Alvez et al. [22] applied ATP-based diagnosis directly to SUMO: for each axiom α, attempt to prove $\neg\alpha$ from the remaining axioms. Success indicates that α contradicts the rest of the knowledge base. This methodology re-engineered approximately 88% of SUMO, finding incorrectly defined axioms, redundancies, and non-desirable properties.

Limitation: Catches logical inconsistency but not type errors, behavioral bugs, or evidential gaps. The methodology is purely deductive and provides no guidance on *which* repair to choose when multiple corrections are possible.

SUMO-GF type checking. Enache [23] translated 17 SUMO modules to GF abstract syntax, using Martin-Löf dependent types to encode SUMO's sort discipline. Of the quantified axioms, **64% passed** type-checking; the remaining 36% broke down as: 12% rejected by the type checker (genuine sort/arity errors), 12% inexpressible because variable types are inferred from usage context, 11% using predicates not translated to GF (`range`, `domain`, `subclass`), and 1% other. This is the direct predecessor of our Stage 1, and the most relevant prior art.

Crucially, examination of the original SUMO-GF source (`Merge.gf`, `MidLevelOntology.gf`, distributed with GF 3.3) reveals that the encoding contains *silent repairs*: classification choices that differ from SUMO's own taxonomy but are not documented as corrections. For example, SUMO declares `Pain` as a subclass of `PathologicProcess` (a `Process`), but the GF grammar declares `fun Pain : Ind EmotionalState` (an `Attribute`). This reclassification makes `contraryAttribute(Pleasure, Pain)` type-correct—a defensible ontological judgment (Pain is a sensation caused by a pathological process, not the process itself)—but it was made implicitly, with no audit trail or canary theorem to verify correctness.

Limitation: Type errors were detected but not automatically repaired; where repairs were made, they were undocumented. The project provides no systematic methodology for distinguishing genuine SUMO bugs from encoding artifacts, nor for verifying that silent repairs preserve the intended semantics.

The gap. No existing system provides a *closed-loop* repair methodology for full-FOL ontologies that (1) detects errors across multiple levels, (2) uses structured signals to *disambiguate* among repair candidates, (3) verifies repairs via proven invariants, and (4) converges. Our proposal addresses this gap.

6.2 The Convergent Repair Loop

We propose a six-phase loop that proceeds top-down through SUMO's class hierarchy, starting from `Entity`:

1. **Formalize.** Translate the next layer of SUMO axioms to GF abstract syntax (Stage 1 of the pipeline). Proceed top-down: first `Entity` and its immediate subclasses, then `Physical`/`Abstract`, and so on.

2. **Detect.** Run the pipeline diagnostics:
   - GF type checker: arity and sort errors.
   - OSLF synthesis: behavioral type violations ($\Diamond/\Box$ ill-formedness).
   - WM evaluator: zero-evidence assertions, scope ambiguities.

3. **Disambiguate.** For each detected error, the pipeline provides structured context that narrows the space of candidate repairs:
   - *GF type context*: bidirectional type inference gives the expected sort at the error site (e.g., "expected `Attribute`, got `Process`"), directly suggesting which argument or declaration is wrong.
   - *OSLF behavioral types*: different repairs produce different operational semantics; the one preserving intended reduction behavior is preferred.
   - *WM evidence scores*: repairs can be ranked by evidential groundability—the fix best supported by evidence is preferred.

4. **Repair.** Propose a minimal KIF patch. Following the axiom-weakening principle [19], weakening is preferred over deletion: strengthen a sort constraint, split a concept, or add a missing argument—rather than removing an axiom entirely.

5. **Verify.** Prove a *canary theorem* (Definition 6.1) that encodes the intended consequence of the repaired axiom. If the canary fails, the repair is incorrect and the loop returns to step 3.

6. **Continue.** Add the canary to the growing regression suite and proceed to the next layer.

Principle 6.1 (Convergence)**.** The loop converges because: (a) each SUMO layer has finitely many axioms; (b) each successful repair strictly reduces the error count at that layer; (c) canary theorems prevent regression (a repair at layer k that breaks a canary at layer $j < k$ is immediately caught). The canary suite grows monotonically, providing an ever-stronger invariant.

Remark 6.1 (Connection to autoformalization)**.** The repair loop has the same structure as the LLM autoformalization loop [27]: formalize $\rightarrow$ check $\rightarrow$ error feedback $\rightarrow$ retry. The key difference is the *richness of the error signal*: where autoformalization typically receives only a type error message, our pipeline provides type context, behavioral diagnostics, and evidence scores—three independent disambiguation channels.

6.3 Canary Theorems

Definition 6.1 (Canary theorem)**.** A *canary theorem* for a repair r applied to axiom α is a small, proven proposition γ_r such that:

1. γ_r is a direct intended consequence of the repaired axiom $\alpha[r]$.
2. γ_r is *falsified* by any incorrect repair r' (i.e., $\alpha[r'] \not\models \gamma_r$).
3. γ_r is proven in the proof assistant (Lean), not merely tested via ATP query.

Example 6.1 (Classmate arity canary)**.** After repairing `classmate` from `IrreflexiveRelation` membership to an explicit ternary irreflexivity axiom (§9), the canary is:

$$\gamma_{\texttt{classmate}} : \forall\, a\, c.\ \neg\, \texttt{classmate}(a, a, c)$$

This is provable from the repaired axiom. If a future edit accidentally changes the arity (e.g., to quaternary), the canary immediately fails.

Example 6.2 (Pain/Attribute canary)**.** After splitting `Pain` into `PainProcess` (a subclass of `Process`) and `PainSensation` (an `Attribute`), the canary suite includes:

$$\begin{aligned}
\gamma_1 &: \texttt{instance}(\texttt{PainProcess}, \texttt{Process}) \\
\gamma_2 &: \texttt{instance}(\texttt{PainSensation}, \texttt{Attribute}) \\
\gamma_3 &: \texttt{contraryAttribute}(\texttt{Pleasure}, \texttt{PainSensation}) \text{ is well-typed} \\
\gamma_4 &: \texttt{PainProcess} \neq \texttt{PainSensation}
\end{aligned}$$

Any repair that merges them back or assigns the wrong metaclass breaks at least one canary.

Canary theorems relate to Keet and Ławrynowicz's white-box testing [7]: both generate test obligations from axiom structure. The key difference is that canaries are *proven* in a proof assistant, not queried via a semi-decidable ATP call. A canary either holds or it does not—there is no timeout ambiguity.

6.4 Disambiguation via Behavioral Evidence

The central advantage of the pipeline for repair—as opposed to purely type-directed or purely deductive approaches—is that it provides *multiple independent signals* for choosing among repair candidates. We illustrate with the Pleasure/Pain bug (§3).

The ambiguity. The type error is clear: `contraryAttribute(Pleasure, Pain)` requires `Pain` to be an `Attribute`, but SUMO's taxonomy says `Pain` is a `Process`. Three repairs are possible:

(a) Reclassify `Pain` as `Attribute` (breaking its process axioms).
(b) Generalize `contraryAttribute` to accept `Process` (weakening the sort constraint).
(c) Split: create `PainSensation` : `Attribute` alongside the existing `Pain` : `Process` (concept refinement).

Type signal alone. A pure type checker (like SUMO-GF [23]) reports the mismatch but cannot distinguish (a), (b), and (c). All three restore type-correctness.

Behavioral signal. OSLF synthesis reveals that `Pain` participates in reduction rules (it has an operational semantics as a process: onset, duration, intensification, subsidence). Repair (a) would orphan these rules, producing $\Box$-violations for any scenario involving pain dynamics. Repair (b) would allow `contraryAttribute` to compare processes and attributes, producing ill-typed behavioral consequences. Repair (c) preserves all existing reduction rules for `Pain`/`PainProcess` while creating a new, behaviorally inert `PainSensation` attribute. *The behavioral signal selects (c).*

Evidence signal. The WM evaluator can further rank candidates: medical and neuroscience corpora provide strong evidence for Pain-as-Process ($\langle n^+, n^- \rangle$ with high n^+), while folk-psychological usage supports Pain-as-Attribute-like quality. The split repair (c) maximizes total evidence across both readings.

Table 2: Repair disambiguation for the Pain/Attribute bug.

Signal	**(a) Reclassify**	**(b) Generalize**	**(c) Split**
Type-correct?	✓	✓	✓
Behavioral coherence	×	×	✓
Evidence grounding	partial	partial	maximal
Canary suite passes	$\gamma_{1,3}$ fail	γ_3 fails	all pass

This multi-signal disambiguation is what distinguishes the pipeline approach from all prior ontology repair methods. Type-directed repair (SUMO-GF, LeanSUMO) narrows the space; behavioral types eliminate operationally incoherent candidates; evidence scores rank the remainder.

7 The Evidence Model: Structured Assurance for Repair Decisions

Detection alone does not justify a repair. The gap between "`.unsat` detected" and "repair safely applicable" requires structured evidence, multi-agent review, and quantitative assurance scoring. This section describes the *Evidence Model*—a formal framework implemented in Lean 4 (`EvidenceModel.lean`, 282 lines)—that governs which repairs may be applied and when.

7.1 Repair Archetypes and Beta Priors

Not all repairs carry the same epistemic risk. A syntax fix (`exist`→`exists`) is nearly certain; reclassifying a concept across ontological branches requires deep judgment. We define six *repair archetypes*, each with a Beta-distribution prior (α_0, β_0) encoding how much evidence is needed before acting:

Archetype	α_0	β_0	**Example**
`syntaxArity`	12.0	1.0	`exist`→`exists`, missing args
`argSwapTyped`	10.0	1.5	D24: agent arguments swapped
`domainNarrowing`	7.0	3.0	D26: Group→AutonomousAgent
`hierarchyGap`	6.0	3.5	D23: Artifact bypasses SelfConnectedObject
`ontologyReclassification`	4.0	4.0	D1: Pain Process→Attribute
`worldSemanticClaim`	3.0	5.0	Deep semantic/world-model judgment

Higher α_0 relative to β_0 means the archetype is easier to trust from less evidence. Syntax fixes have prior mean $12/13 \approx 0.92$; world-semantic claims have prior mean $3/8 = 0.375$—starting from skepticism.

7.2 Evidence Atoms and Source Weighting

Every piece of evidence—from a proven-sound `checkLang` result to a literature citation to an LLM review vote—is normalized into a common *evidence atom* schema carrying: claim ID, decision ID, archetype, direction (supports/contradicts), source type, formal expression, strength, confidence, independence group (to avoid double-counting correlated evidence), and a reproducibility hash.

Source types are weighted by reliability:

Source type	Weight	Rationale
`checkLang` (proven sound)	1.00	Verified by `checkLangUsing_sat_sound`
`stratumScan`	0.75	`constraint_check.py` output
`kifCitation`	0.65	Direct KIF file:line reference
`engGFParsed`	0.60	GF runtime: unique parse tree + roundtrip verified
`humanReview`	0.60	Human expert vote
`kifRendered`	0.50	Deterministic KIF → controlled English template
`docGFParsed`	0.50	Doc definition: GF-parsed + KIF cross-checked
`literature`	0.45	Encyclopedia, medical standard, etc.
`llmReview`	0.35	Claude Code / Codex / GPT-5.2 Pro vote
`englishDoc`	0.30	Corroborative hint from `(documentation ...)`

The `checkLang` source type has maximal weight because its results are *proven sound*: Theorem `checkLangUsing_sat_sound` guarantees that a `.sat` result implies the semantic property actually holds. The three GF-derived sources (`kifRendered`, `engGFParsed`, `docGFParsed`) form an ascending trust hierarchy: template rendering (deterministic but pattern-limited), GF runtime parsing (typed grammar with AST equivalence verification), and documentation cross-checking (GF-parsed definitions verified against KIF facts). LLM reviews have low weight due to correlation (multiple LLMs trained on overlapping data). `englishDoc` is the weakest source: prose-level hints from SUMO documentation strings, used only as corroborative evidence and never as sole authority.

7.3 Assurance Scoring

Evidence atoms contribute weighted pseudo-counts to the archetype's Beta prior. A supporting atom from source s with strength t and confidence c adds $w_s \cdot t \cdot c$ to α; a contradicting atom adds the same to β. The *assurance number* ("ass-number") is the lower bound of the 95% credible interval of $\text{Beta}(\alpha, \beta)$:

$$\text{ass} = F^{-1}_{\text{Beta}(\alpha,\beta)}(0.025)$$

where F^{-1} is the inverse CDF. This is a conservative estimate: we are 97.5% confident the true "goodness" of the repair exceeds ass.

Thresholds.

- ass ≥ 0.98: eligible for auto-application (if review quorum met)
- $0.93 \leq$ ass < 0.98: hold for additional reviewer
- ass < 0.93: remain logged/deferred

7.4 Multi-Agent Review Protocol

Classification decisions for full-FOL ontologies benefit from diverse perspectives. The Evidence Model supports a multi-agent review protocol where each reviewer (human or AI) submits a *review*

vote: decision ID, reviewer identity, verdict (approve/reject/abstain), confidence, and hashes of the rationale and evidence bundle reviewed.

Planned reviewers for the pipeline (review votes not yet populated; all 26 decisions are at `pendingReview`):

- **Claude Code**: primary development agent, produces initial evidence and repair proposals
- **Codex**: independent review with deep context; identified D20/D21 status corrections and evidence policy gaps
- **GPT-5.2 Pro**: cross-validation with different training distribution
- **Human**: final authority, especially for world-semantic judgments

LLM reviewers in the same independence group (e.g., all GPT-family models) have their evidence down-weighted to avoid inflating confidence from correlated sources.

Review lifecycle. Each repair decision transitions through states: `pendingReview` → `reviewApproved` → `applied` → `verified` (or → `rejected` if strong contradictory evidence emerges). No KIF or SUMO-GF repairs are applied until the decision reaches `reviewApproved`.

7.5 WM Witnesses: Persisting checkLang Results

The comprehensive audit (§B.7) produces `checkLang` results at build time. The Evidence Model persists these as structured *WM witnesses* in the repair log, each recording: witness ID, query, expected result, observed result, pass/fail, and source reference.

For decisions D22–D26, the current witness inventory is:

ID	Decision	Query (reachabilityTest)	Exp.	Obs.	Pass
D22	LinguisticExpression$\not\to$Object	LE → object	unsat	unsat	✓
D23	Artifact$\not\to$SelfConnObj	Artifact → selfconnobj	unsat	unsat	✓
D24a	CogAgent$\not\to$Process (pre)	CA → process	unsat	unsat	✓
D24b	CogAgent→Object (post)	CA → object	sat	sat	✓
D26a	CogAgent$\not\to$Group (pre)	CA → group	unsat	unsat	✓
D26b	Group→Object (positive)	Group → object	sat	sat	✓

All 6 witnesses pass. Combined with the 14 existing testable decisions from D1–D20, the comprehensive audit now covers **20 passed / 0 failed / 8 N/A** across 26 decisions. Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/data/wm_witnesses.jsonl`, `Mettapedia/Languages/GF/SUMO/data/de` `Mettapedia/Languages/GF/SUMO/export_wm_witnesses.py`.

7.6 Evidence-Quantale Compatibility

The Evidence Model's (n^+, n^-) pseudo-counts align directly with `Mettapedia.Logic.EvidenceQuantale`, which models evidence as pairs in $\mathbb{R}^\infty_{\geq 0} \times \mathbb{R}^\infty_{\geq 0}$ forming a quantale under component-wise addition. The conversions `toSTV`/`ofSTV` bridge to PLN's `SimpleTruthValue` (strength, confidence). This means the repair pipeline's evidence atoms can be consumed by PLN inference—e.g., to propagate evidence about a concept's classification to axioms that use it.

Policy: no repairs without evidence. The critical policy decision is: `autoFixable` marks a candidate *eligible* for auto-application, not "apply now." Application requires: (1) sufficient evidence atoms across required witness types (pre-fix, post-fix, regression), (2) assurance number $\geq$ threshold, and (3) review quorum met. This prevents premature repairs from propagating errors deeper into the ontology.

7.7 GF Runtime Evidence Pipeline

Beyond proven-sound `checkLang` witnesses and manual KIF citations, we construct evidence atoms *automatically* from SUMO's own axioms and documentation using the Grammatical Framework (GF) as a typed grammar runtime. This yields three additional evidence lanes, each with distinct trust characteristics.

Lane A: Template-rendered controlled English (`kifRendered`). A deterministic template renderer converts each KIF axiom into controlled English matching GF linearization patterns: `(subclass X Y)` ↦ "`X is a subclass of Y`", `(instance X Y)` ↦ "`X is an instance of Y`", and so on through 22 relation patterns covering quantifiers (`forall`, `exists`), connectives (`if...then`, `iff`, `and`, `or`, `not`), and thematic roles (`agent`, `patient`, `instrument`). Running this over the 937 repair-relevant axioms produces 10,584 template renderings, all with 100% AST roundtrip fidelity (the rendered English, when parsed back through the template grammar, reconstructs the original KIF AST). This lane contributes 805 evidence atoms at weight 0.50.

Lane B: GF runtime parsing (`engGFParsed`). A minimal RGL-free GF grammar (`AxiomCE.gf`) encodes the controlled English patterns as typed abstract syntax:

```
cat Stmt ; Formula ; Term ; ArgN ;
fun SubclassStmt  : Term -> Term -> Stmt ;
    InstanceStmt  : Term -> Term -> Stmt ;
    ExistsFml     : Term -> Formula -> Formula ;
    ImplFml       : Formula -> Formula -> Formula ;
    ...
```

with an auto-generated closed lexicon of 17,718 terms (all identifiers appearing in SUMO axioms and documentation cross-references). The grammar compiles to a PGF (Portable Grammar Format) file and runs as a GF subprocess.

For each axiom, the pipeline: (1) normalizes the template-rendered English (CamelCase → underscore, comma spacing); (2) parses through GF (`p -cat=Stmt`); (3) linearizes the parse tree back (`l`); and (4) verifies *AST equivalence*: the GF tree's top-level constructor must match the KIF head relation (e.g., `SubclassStmt` for `(subclass ...)`). Only axioms passing all three gates—unique parse, string roundtrip, and AST equivalence—produce `engGFParsed` atoms.

Disambiguation. A key engineering decision is the separation of *canonical* (machine) English from *human-friendly* English. The canonical form parenthesizes every compound sub-formula: `if ( A and ( B and C ) ) , then ( D )` — with a grammar constructor `ParenFml` producing `` ``(`` ++ f.s ++ ``)`` ``. N-ary conjunction right-associates: `A and ( B and C )` rather than the ambiguous `A and B and C`. This eliminates all parse ambiguity: the GF chart parser returns exactly one tree for every parenthesized axiom.

Results. Of 937 repair-relevant axioms: 802 (85.6%) parse uniquely with roundtrip and AST match; 0 parse ambiguously; 135 fail to parse. The 135 failures reflect missing grammar coverage for SUMO-specific constructs (checkpoint 2026-02-23):

Gap family	Count
Complex connectives (nested if/for-all)	47
Multi-variable existentials (∃2)	20
Multi-variable existentials (∃3)	16
Statement reification (`KappaFn`)	16
Multi-variable existentials (∃4+)	15
Spatial verbs (`ends_up_at`, etc.)	6
Row-variable predicates (`@ROW`)	5
IFF at top level (`iff_toplevel`)	5
Obliged-perform constructs	3
Other	2

These are grammar coverage gaps, not GF expressiveness limits. Grammar improvements between v1 and this checkpoint resolved: functional term equality (`equal`→"is equal to"), 16 new verb/relation patterns, and formula-argument parenthesization for higher-order relations (`holdsDuring`, `believes`, `desires`). The aggregate count reflects SUMO's large vocabulary of specialized relations. This lane contributes 514 evidence atoms at weight 0.60. Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/data/axioms_gf_parse.jsonl`, `Mettapedia/Languages/GF/SUMO/data/`
`Mettapedia/Languages/GF/SUMO/data/gf_parsed_evidence_atoms.jsonl`, `Mettapedia/Languages/GF/SUMO/s`

Lane C: Documentation parsing (`docGFParsed`). SUMO's `(documentation Term EnglishLanguage ``...'`` entries contain encyclopedia-style definitions. We extract definitional head claims using four regex patterns:

1. "A $X is a $Y" or "$X is a $Y" → subclass claim
2. "$X is an instance of $Y" → instance claim
3. "Any $X that…" → implied subclass of X's parent
4. "A $X that…" → typed particular (implied subclass)

Extracted heads are normalized and parsed through the same `AxiomCE` grammar. A KIF cross-check classifies each parsed claim as *consistent* (claim matches a KIF `subclass`/`instance` fact), *contradicts* (KIF has a different superclass for the same subject), or *no_kif_data* (no matching KIF fact found).

Results. Of 13,832 documentation entries: 2,130 yield extractable head claims; 1,555 of these parse uniquely through GF (73%). KIF cross-checking: 425 consistent, 225 contradictory, 905 no KIF data. This lane contributes 60 evidence atoms at weight 0.50, covering 8 repair decisions. Contradictory atoms (where documentation disagrees with KIF) receive reduced strength (0.40) and confidence (0.35), reflecting the lower reliability of prose-level descriptions vs. formal axioms.

Measurement provenance note. All counts/percentages in this subsection come from repository-local scripts and logs in the reported snapshot; they are engineering measurements, not Lean theorems. Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/data/doc_gf_parse.jsonl`,
`Mettapedia/Languages/GF/SUMO/data/doc_gf_parsed_evidence_atoms.jsonl`, `Mettapedia/Languages/GF/SUM`
`Mettapedia/Languages/GF/SUMO/sumo_doc_gf_parse.py`, `Mettapedia/Languages/GF/SUMO/sumo_doc_to_evide`

Five-lane evidence architecture. The full evidence model used in this pipeline slice draws from five operationally independent lanes:

Lane	Weight	Independence group	Atoms
WM witnesses (`checkLang`)	1.00	`wm.*`	6
KIF rendered templates (`kifRendered`)	0.50	`kif.*`	805
GF runtime parsed (`engGFParsed`)	0.60	`gfparse.*`	514
Doc GF parsed (`docGFParsed`)	0.50	`docgf.*`	60
Doc English hints (`englishDoc`)	0.30	`doc.*`	1,166
Total			**2,551**

Independence groups prevent double-counting within and across lanes: each group retains only the highest-weight atom. Of the 26 repair decisions, 14 now have evidence atoms from at least two independent lanes, enabling meaningful Beta aggregation. The `SumoNTT` regression suite remains at 20 passed, 0 failed, 8 N/A (encoding choices and meta-decisions not testable as reachability queries). Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/data/pipeline_audit.json`, `Mettapedia/Langua` `Mettapedia/Languages/GF/SUMO/data/gf_parsed_evidence_atoms.jsonl`, `Mettapedia/Languages/GF/SUMO/d` `Mettapedia/Languages/GF/SUMO/data/doc_evidence_atoms.jsonl`.

8 Comparison with Direct SUMO → Lean

We developed two direct approaches before arriving at the pipeline.

8.1 sumo-lean4: KIF Checker and HOL Emitter

In the reported snapshot, the `sumo-lean4` tool parses all 113 KIF files and:

1. Checks arity consistency (relation usage vs. declaration).
2. Emits "HOL-native" Lean code with curried predicates for fixed-arity relations and `List`-based application for variable-arity ones.

After one pass in that snapshot, the checker reported "no issues found" across all files (after applying the fixes in §3).

Strength: Found genuine arity and argument bugs quickly.

Limitation: The emitted Lean code declares SUMO axioms as Lean `axiom`s—opaque to Lean's type system. Once imported, bugs *become trusted axioms in Lean's kernel.* There is no behavioral or evidential checking.

8.2 LeanSUMO: Type-Based Attributes

The LeanSUMO project encoded SUMO concepts as Lean structures with attribute fields:

```
structure Human extends Organism where
  sex : SexAttr
structure Man extends Human where
  isMale : sex = SexAttr.Male
```

This enabled elegant 4-line disjointness proofs (Man ≠ Woman) and caught the Pleasure/Pain type confusion at compile time.

Strength: Lean's type system catches sort errors as type mismatches. Proofs are trivial.

Limitation: Each SUMO class requires manual encoding as a Lean structure. With 20 000+ classes, this does not scale. No automation exists for the KIF → structure translation. And crucially, there is no behavioral semantics: the encoding captures *what things are* but not *how they behave.*

8.3 The Pipeline Approach

The pipeline addresses both limitations:

- **Automation**: the GF grammar is generated from SUMO's `domain/range/subclass` declarations, just as the HOL emitter already does. No manual per-class encoding.
- **Behavioral semantics**: OSLF synthesis derives $\Diamond/\Box$ types automatically from operational rules.
- **Evidence grounding**: the WM layer quantifies how well-supported each assertion is.
- **Sort safety**: GF categories provide the same sort discipline as LeanSUMO's typed attributes, but generated.

Table 3: Bug classes caught by each approach.

Bug class	sumo-lean4	LeanSUMO	Pipeline
Arity mismatch	✓	✓	✓
Missing arguments	✓		✓
Type/sort confusion		✓	✓
Semantic contradiction			✓
Behavioral inconsistency			✓
Ungroundable assertion			✓
Scope ambiguity			✓
Compositional bug			✓
Repair disambiguation			✓

9 Worked Example: The Classmate Arity Bug

We trace a real SUMO bug through the pipeline.

The bug. SUMO declares:

```
(instance classmate IrreflexiveRelation)
(domain classmate 1 Human)
(domain classmate 2 Human)
(domain classmate 3 Organization)
```

But `IrreflexiveRelation` is defined in `Merge.kif` as a subclass of `BinaryRelation`. A ternary relation cannot be an instance of a binary-only relation class.

Stage 1 (sumo → gf). The GF grammar generator creates:

```
cat Human ; cat Organization ;
fun classmate : Human -> Human -> Organization -> Prop ;
```

and separately:

```
fun IrreflexiveRelation_inst :
  (Entity -> Entity -> Prop) -> Prop ;
```

The assertion `(instance classmate IrreflexiveRelation)` requires passing `classmate` (a ternary function) to `IrreflexiveRelation_inst` (which expects a binary function). *The GF type checker rejects this: type mismatch.*

Stage 2 (gf → oslf). Had the bug not been caught at Stage 1, the OSLF synthesis would derive rewrite rules for `classmate` with three arguments. The irreflexivity axiom `(forall (?R) (=> (instance ?R IrreflexiveRelation) (not (?R ?X ?X))))` would apply to the binary projection of `classmate`, producing $\Box(\neg\texttt{classmate}(a, a))$—but this is ill-formed because `classmate` requires three arguments. The $\Box$ evaluation would flag an arity error in the behavioral type.

Stage 3 (oslf → wm). The evidence evaluator would assign $\langle 0, 0\rangle$ (zero evidence) to the irreflexivity claim for `classmate`, since the claim is about a binary usage that never occurs in the ontology's operational semantics. This zero-evidence diagnostic would flag the assertion as ungroundable.

Comparison.

- **KIF checker**: catches the arity mismatch directly (our `sumo-lean4` found this).
- **ATP**: may or may not notice, depending on how the ternary/binary conflict interacts with other axioms.
- **OntoClean**: irrelevant (this is a syntactic/semantic bug, not a metaproperty issue).
- **Keet testing**: would generate test cases for irreflexivity, which might succeed or fail depending on the ATP's handling of the arity conflict.
- **Pipeline**: catches at Stage 1 (type error), Stage 2 ($\Box$ ill-formedness), and Stage 3 (zero evidence). Three independent diagnostics from different layers.

10 Related Work and Discussion

Faithful HOL embeddings. Benzmüller et al. [13] embed various logics (modal, deontic, conditional) into classical HOL, enabling ATP tools to reason about non-classical ontological assertions. Brown, Pease, and Urban [24] extended this by embedding the SUMO-K fragment into higher-order set theory via Isabelle/HOL, providing formal semantics for SUMO's higher-order features. Our pipeline is complementary: where these embeddings preserve the *deductive* content of axioms, our pipeline extracts their *operational* content. Combining both—deductive verification via faithful embedding plus behavioral verification via OSLF—would provide broader end-to-end coverage across deductive and behavioral checks.

Ontology repair. Troquard et al. [19] introduced axiom weakening as an alternative to deletion-based repair, using refinement operators from inductive logic programming. Baader and Kriegel [20, 21] established a formal correspondence between optimal repair and AGM belief revision (pseudo-contraction), with practical algorithms for $\mathcal{EL}$ TBoxes. Alvez et al. [22] applied ATP-based diagnosis specifically to SUMO, re-engineering 88% of the ontology. Our repair loop (§6) extends this line of work to full FOL with behavioral and evidential disambiguation signals that these approaches lack.

Autoformalization. The repair loop's structure—formalize, check, use error feedback to correct, retry—mirrors the LLM autoformalization loop [27], where a language model generates formal statements and a proof assistant provides verification feedback. The key difference is the richness of the feedback: our pipeline provides type context, behavioral diagnostics, and evidence scores, whereas autoformalization typically receives only type error messages.

Modular ontologies. Stuckenschmidt et al. [14] develop theory and techniques for decomposing large ontologies into modules with well-defined interfaces. Our GF-based approach naturally supports modularity: each SUMO domain file becomes a GF module, and the OSLF synthesis respects module boundaries.

Connection to Mettapedia. The referenced GF → OSLF → WM pipeline slice is formalized in the Mettapedia Lean 4 library and currently builds without `sorry`/axiom placeholders in that slice. The PLN evidence quantale, the Galois connection, the compositional semantics, and the world-model bridge are represented there by theorem-level endpoints. SUMO-GF is a new concrete syntax for this existing, verified infrastructure, and the Evidence Model (`EvidenceModel.lean`) extends the repair log with structured assurance scoring that connects directly to `EvidenceQuantale`'s (n^+, n^-) algebra.

Limitations.

- The pipeline does not replace OntoClean-style adequacy judgments. Philosophical metaproperties (rigidity, identity) require human expertise.
- The behavioral-correctness layer (§2.4) presupposes an operational reading of the ontology. For purely classificatory fragments without process axioms, Stages 2–3 add little value.
- The SUMO → GF translation (Stage 1) is not yet implemented as an automated tool; it is described as a well-motivated extension of the existing `sumo-lean4` HOL emitter.
- Row variables in KIF require special handling (bounded unfolding or higher-order GF categories).
- The formalization-driven repair loop (§6) is partially implemented: the detection pipeline, evidence model, and multi-agent review infrastructure are in place, with 26 decisions logged and 20 passing formal verification. The repair-selection heuristics and the top-down traversal strategy require further engineering effort and empirical validation on SUMO's full 20k axioms. The `score_repairs.py` tool to compute assurance numbers automatically per decision is planned but not yet built.

Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/RepairLog.lean`, `Mettapedia/Languages/GF/SUM`
`Mettapedia/Languages/GF/SUMO/data/decision_coverage.json`, `Mettapedia/Languages/GF/SUMO/data/revi`

11 Conclusion

Ontology correctness is not a single property but a hierarchy: syntactic well-formedness, semantic consistency, taxonomic adequacy, and behavioral correctness. Existing methods address the first three layers but leave behavioral adequacy—whether an ontology's axioms produce the intended operational consequences—unexamined.

The pipeline SUMO → GF → OSLF → WM provides the missing layers:

- GF's typed abstract syntax catches sort and arity errors at grammar level.
- OSLF's behavioral types ($\Diamond$, $\Box$, native types) detect operational inconsistencies.
- The world-model's evidence-valued semantics flags ungroundable assertions, contradictory evidence flows, and scope ambiguities.

Beyond detection, the formalization-driven repair loop (§6) provides a principled methodology for *correcting* ontology bugs: multi-level diagnostics disambiguate among repair candidates, canary theorems prevent regression, and the loop converges by construction. The Evidence Model (§7) adds quantitative assurance: Beta-distribution scoring with archetype-specific priors, multi-agent review (Claude Code, Codex, GPT-5.2 Pro, human), and PLN-compatible evidence atoms that integrate with the proven evidence quantale. No repair is applied until structured evidence, assurance

thresholds, and review quorum are satisfied. This addresses the gap left by SUMO-GF [23], whose type checker rejected 12% of axioms outright (with a further 24% inexpressible), and which made silent classification repairs with no audit trail or verification methodology.

The GF → OSLF → WM core is formalized in Lean 4 in the referenced pipeline slice, with no `sorry`/axiom placeholders in that slice. Extending the pipeline to SUMO—a large, openly available formal ontology—is the natural next step: it inherits the benefits of both our `sumo-lean4` arity checker and LeanSUMO's type-based attributes, while adding behavioral and evidential correctness layers that neither provides. As a proof of concept, we encode the FOET-relevant fragment of SUMO-GF and run it through the full pipeline (Appendix B), automatically detecting known bugs including the Pain/Attribute type confusion. The repair loop transforms the pipeline from a static analysis tool into an interactive ontology engineering methodology.

References

[1] A. Pease, I. Niles, and J. Li, "The Suggested Upper Merged Ontology: A Large Ontology for the Semantic Web and its Applications," in *Working Notes of the AAAI-2002 Workshop on Ontologies and the Semantic Web*, 2002.

[2] A. Pease, "The Annual SUMO Reasoning Prizes at CASC," *CADE Workshop*, 2015.

[3] A. Pease and G. Sutcliffe, "First Order Reasoning on a Large Ontology," in *Proc. ESARLT*, pp. 61–70, 2007.

[4] A. Pease, "Reasoning and Language Generation in the SUMO Ontology," *Technical Report*, 2019.

[5] P. Hayes and C. Menzel, "A Semantics for the Knowledge Interchange Format," in *Proc. IJCAI Workshop on the KIF Standard*, 2001.

[6] A. Kalyanpur, B. Parsia, E. Sirin, and J. Hendler, "Debugging Unsatisfiable Classes in OWL Ontologies," *Journal of Web Semantics*, vol. 3, pp. 268–293, 2005.

[7] C. M. Keet and A. Ławrynowicz, "Automatic White-Box Testing of First-Order Logic Ontologies," *arXiv:1705.10219*, 2017.

[8] C. M. Keet and A. Ławrynowicz, "Black-Box Testing of First-Order Logic Ontologies Using WordNet," *arXiv:1705.10217*, 2017.

[9] N. Guarino and C. A. Welty, "An Overview of OntoClean," in *Handbook on Ontologies*, Springer, pp. 151–159, 2009.

[10] A. Gangemi and V. Presutti, "Ontology Design Patterns," in *Handbook on Ontologies*, Springer, pp. 221–243, 2009.

[11] A. Ranta, "Grammatical Framework: A Type-Theoretical Grammar Formalism," *Journal of Functional Programming*, vol. 14, no. 2, pp. 145–189, 2004.

[12] A. Ranta, K. Angelov, and R. Enache, "Typeful Ontologies with Direct Multilingual Verbalization," in *Proc. Controlled Natural Language*, 2012.

[13] C. Benzmüller, "Faithful Logic Embeddings in HOL—Deep and Shallow," *arXiv:2502.19311*, 2025.

[14] H. Stuckenschmidt, C. Parent, and S. Spaccapietra, Eds., *Modular Ontologies: Concepts, Theories and Techniques for Knowledge Modularization*, LNCS 5445, Springer, 2009.

[15] B. Goertzel, M. Iklé, I. F. Goertzel, and A. Heljakka, *Probabilistic Logic Networks: A Comprehensive Framework for Uncertain Inference*, Springer, 2008.

[16] T. Bittner, "On the Computational Realization of Formal Ontologies," *Technical Report*, University at Buffalo, 2004.

[17] J. Euzenat and P. Shvaiko, *Ontology Matching*, 2nd ed., Springer, 2013.

[18] A. Pease, G. Sutcliffe, N. Siegel, and S. Trac, "Translating SUMO-K to Higher-Order Set Theory," 2010.

[19] N. Troquard, R. Confalonieri, P. Galliani, R. Peñaloza, D. Porello, and O. Kutz, "Repairing Ontologies via Axiom Weakening," in *Proc. AAAI*, pp. 1981–1988, 2018.

[20] F. Baader and F. Kriegel, "Optimal Repairs in Ontology Engineering as Pseudo-Contractions in Belief Change," in *Proc. ACM SAC*, pp. 846–855, 2023.

[21] F. Baader and F. Kriegel, "Optimal Fixed-Premise Repairs of $\mathcal{EL}$ TBoxes," in *Proc. KI*, LNCS 13404, pp. 103–117, Springer, 2022.

[22] J. Alvez, P. Lucio, and G. Rigau, "Adimen-SUMO: Reengineering an Ontology for First-Order Reasoning," *International Journal on Semantic Web and Information Systems*, vol. 8, no. 4, pp. 80–116, 2012.

[23] R. Enache and K. Angelov, "Typeful Ontologies with Direct Multilingual Verbalization," in *Proc. Controlled Natural Language*, LNCS 7427, pp. 52–67, Springer, 2012.

[24] C. E. Brown, A. Pease, and J. Urban, "Translating SUMO-K to Higher-Order Set Theory," in *Proc. FroCoS*, LNCS 14279, pp. 253–274, Springer, 2023.

[25] M. Horridge, B. Parsia, and U. Sattler, "Laconic and Precise Justifications in OWL," in *Proc. ISWC*, LNCS 5318, pp. 323–338, Springer, 2008.

[26] S. Schlobach and R. Cornet, "Non-Standard Reasoning Services for the Debugging of Description Logic Terminologies," in *Proc. IJCAI*, pp. 355–362, 2003.

[27] Y. Wu, Z. Jiang, et al., "Autoformalization in the Era of Large Language Models: A Survey," *arXiv:2505.23486*, 2025.

[28] B. Goertzel and Z. Kothare, "The Formal Ethics Seed Ontology (FOET)," *Garden of Minds*, 2022. https://gardenofminds.art/ethics/formal-ethics-seed-ontology/

[29] C. Benzmüller, A. Fuenmayor, and D. Fuenmayor, "Formalisation and Evaluation of Alan Gewirth's Proof for the Principle of Generic Consistency in Isabelle/HOL," *Archive of Formal Proofs*, 2018.

A Case Study: SUMO Dependencies of the Formal Ethics Seed Ontology

To make the repair loop concrete, we identify the *exact* SUMO fragment required by a real downstream application: the Formal Ethics Seed Ontology (FOET) [28], a 6 853-line KIF formalization of four ethical paradigms (deontological, value-judgment, utilitarian, virtue ethics) with inter-paradigm translations and a Lean 4 formalization including a fully proven Gewirth PGC argument [29]. This appendix traces FOET's SUMO dependencies upward through the class hierarchy, producing a minimal subontology that must be fully debugged and repaired before the pipeline can verify FOET end-to-end.

A.1 FOET's SUMO Concept Inventory

The FOET ontology introduces domain-specific classes (left column) that are positioned within SUMO's existing hierarchy (right column). Table 4 lists the key concept groups.

Table 4: FOET concepts and their SUMO superclass anchors.

FOET concept	**Direct superclass**	**SUMO anchor**
Ethical attributes		
`MoralValueAttribute`	`MoralAttribute`	`NormativeAttribute`
`DeonticAttribute`	`ObjectiveNorm`, `MoralAttribute`	`NormativeAttribute`
`VirtueAttribute`	`MoralVirtueAttribute`	`PsychologicalAttribute`
`ViceAttribute`	`MoralVirtueAttribute`	`PsychologicalAttribute`
Agents and processes		
`VirtuousAgent`	`AutonomousAgent`	`Object`
`AutonomousAgentProcess`	`Process`	`Physical`
`VirtuousAct`	`AutonomousAgentProcess`	`Process`
`ChoicePoint`	`Set`, `NonNullSet`	`SetOrClass`
`Situation`	(direct)	`Physical`
Propositions and theories		
`ActionFormula`	`Formula`	`Sentence`
`Conjecture`	`Sentence`	`LinguisticExpression`
`EthicalTheory`	(set of `EthicalSentence`)	`Proposition`
`Argument`	`Proposition`	`Abstract`
Values (Schwartz–Haidt)		
`Value`	(direct)	`Abstract`
10 Schwartz values	`Value`	`Abstract`
Utility		
`UtilityFormulaFn`	`UnaryFunction`	`Function`
`Consequentialism`	`Ethics`	domain module

A.2 SUMO Relations Used by FOET

FOET introduces 30+ relations. The most critical (those appearing in axioms with inter-paradigm translations) are:

Relation	SUMO type	Key domains
`hasAttribute`	`BinaryPredicate`	Agent × Attribute
`desires`	`PropositionalAttitude`	Agent × Formula
`holdsValue`	`BinaryPredicate`	Agent × Value
`holdsObligation`	`BinaryPredicate`	Formula × Agent
`realizesFormula`	`BinaryPredicate`	Process × Formula
`capableInSituation`	`QuaternaryPredicate`	ProcessClass × CaseRole × Object × Situation
`situationFn`	`UnaryFunction`	Process → Situation
`influences`	`BinaryPredicate`	(general)
`conjectures`	`PropositionalAttitude`	Agent × Conjecture

A.3 Upward Dependency Graph

Figure 1 shows the SUMO class hierarchy from `Entity` down to FOET's leaf concepts. *Every node on a path from `Entity` to a FOET concept must be correctly formalized* for the pipeline to verify FOET's axioms.

A.4 Repair Scope Estimate

From the dependency graph, we identify three "repair strata" ordered by priority:

1. **Stratum 0 (Upper ontology core)**: `Entity`, `Physical`, `Abstract`, `Object`, `Process`, `Attribute`, `SetOrClass`, `Relation`, `Proposition`. These 9 classes and their inter-relations (subclass, instance, domain, range) form the skeleton. Approximately 200 axioms in Merge.kif. Known bugs: minimal (heavily tested by Adimen-SUMO [22]).

2. **Stratum 1 (Mid-level anchors)**: `Agent` chain (`AutonomousAgent` → `SentientAgent` → `CognitiveAgent`), `NormativeAttribute` → `ObjectiveNorm`, `PsychologicalAttribute`, `Formula/Sentence`, `Predicate/Function`, `PropositionalAttitude`. Approximately 500 axioms. Known bugs: type confusion in the `Attribute` hierarchy (Pain/Pleasure, §3); potential arity issues in `PropositionalAttitude` predicates.

3. **Stratum 2 (FOET-specific)**: `MoralAttribute`, `DeonticAttribute`, `VirtueAttribute`, `AutonomousAgentProcess`, `ChoicePoint`, `Situation`, `Value`, `UtilityFormulaFn`, and all 30+ FOET relations. Approximately 300 axioms (from the FOET KIF file). Known bugs: the FOET KIF itself documented several SUMO compatibility issues in `ResolvedBugsInSUMO.kif`.

Stratum-count provenance (snapshot): `Mettapedia/Languages/GF/SUMO/stratum_scan.py`, `Mettapedia/Languag`
`Mettapedia/Languages/GF/SUMO/stratum1_scan_with_foet_report.txt`, `Mettapedia/Languages/GF/SUMO/str`

Total: approximately **1 000 axioms** spanning 3 strata—a tractable target for the repair loop. This is 5% of SUMO's full 20k axioms, but it covers the upward-closure slice targeted for the ethics-application fragment in this draft.

A.5 Why FOET Is a Good First Target

1. **Lean formalization exists**: the FOET Lean 4 project (27 modules, 4 607 lines) already type-checks with a minimal SUMO kernel (`SumoCore.lean`: `Class`, `instanceOf`, `subclass`). The repair loop would upgrade this kernel to the full 1 000-axiom fragment.
2. **Gewirth PGC is proven**: the culminating theorem (Principle of Generic Consistency) is proven in both Isabelle/HOL and Lean 4 without `sorry`. Canary theorems for the repair loop can be drawn directly from this proof.

3. **Inter-paradigm translations**: the proven loop (Value → Deontic → Utilitarian → Virtue → Value = id) provides a rich set of canary theorems spanning multiple SUMO concept groups.
4. **Practical relevance**: formal ethics for AI alignment requires certified reasoning over normative concepts—exactly the use case where ontology correctness matters most.

B Experiment: SUMO-GF Through the Pipeline

We encode the FOET-relevant fragment of the SUMO-GF abstract syntax [23] in Lean 4 and run it through the full GF → OSLF → WM pipeline, using exactly the same infrastructure as the English and Czech grammars.

B.1 Encoding Strategy: Shallow Flattening

The SUMO-GF abstract syntax uses GF's dependent types:

```
cat Class; El Class; Ind Class; Var Class;
    SubClass (c1,c2 : Class); Inherits Class Class;
    Formula; Stmt;
```

Our pipeline uses flat string-valued sorts. We apply a *shallow flattening*: for each of the ∼50 FOET-relevant SUMO classes C, we generate concrete sort names:

$$\texttt{El}\ C \mapsto \texttt{"El_}C\texttt{"}, \qquad \texttt{Ind}\ C \mapsto \texttt{"Ind_}C\texttt{"}, \qquad \texttt{Var}\ C \mapsto \texttt{"Var_}C\texttt{"}$$

This loses GF's parametric polymorphism but preserves the sort-distinction structure that makes bugs visible, and it is compatible with the existing `gfGrammarLanguageDef` infrastructure without modification.

The 50 classes are drawn from the FOET dependency graph (Appendix A), organized into three strata:

- **Stratum 0** (9 classes): `Entity`, `Physical`, `Abstract`, `Object`, `Process`, `Attribute`, `SetOrClass`, `Relation`, `Proposition`
- **Stratum 1** (18 classes): the Agent chain, normative attributes, psychological attributes, linguistic expressions
- **Stratum 2** (23 classes): FOET-specific concepts (virtues, deontic attributes, choice points, utility functions)

B.2 Pipeline Statistics

Table 5 compares the SUMO-GF encoding with the existing RGL (English/Czech) encoding.

The larger number of sorts and rules reflects SUMO's richer type structure: where the RGL has a single `NP` sort, SUMO-GF has a separate `El_`C for each class C, with explicit coercion functions between them.

B.3 Rewrite Dynamics

We define four classes of rewrite rules, mirroring the GF `data/def` structure:

1. **Reflexive coercion elimination** (∼100 rules): `el_`C`_`$C(x) \Rightarrow x$ for each class C. In GF, `el(C, C, inhz(C), x) = x`.

	RGL (Eng/Cz)	SUMO-GF
Categories (sorts)	25	∼200
Grammar rules	170	∼350
Rewrite rules	14	∼115
Equations	2	50
Galois connection	proven	proven
Native type theory	yes	yes

Table 5: Pipeline statistics: RGL vs. SUMO-GF encoding. The SUMO-GF encoding is larger because each class generates separate sort, coercion, and quantifier entries.

2. **Variable coercion elimination** (∼50 rules): `var_`C`_`$C(x) \Rightarrow x$.
3. **Transitive chain collapse** (11 rules): `el_`B`_`C(`el_`A`_`$B(x)$) $\Rightarrow$ `el_`A`_`$C(x)$ for selected chains $A <: B <: C$.
4. **Double negation**: $\neg\neg\varphi \Rightarrow \varphi$.

All four classes produce verified output from `#eval!` in Lean 4. The reflexive and variable coercions are particularly important: they ensure that $\Diamond$ is *non-vacuous* for identity coercions, meaning the behavioral type system has actual content rather than being trivially satisfied.

B.4 Bug Detection Results

Pain/Attribute type confusion (detected and analyzed). In SUMO's `Mid-level-ontology.kif`, `Pain` is declared `(subclass Pain PathologicProcess)`—a `Process`. Meanwhile, `contraryAttribute` expects arguments of type `El_Attribute`. Our pipeline faithfully follows SUMO's taxonomy:

- `Pain : Ind_Process`
- `Pleasure : Ind_Attribute`
- `contraryAttribute : El_Attribute → El_Attribute → SumoFormula`

The pipeline detects that `Pain` has coercions only to `El_Process → El_Physical → El_Entity`, with *no path* to `El_Attribute`.

Comparison with the original sumo-gf. Examination of `MidLevelOntology.gf` from the GF 3.3 distribution reveals that Enache silently reclassified Pain:

```
fun Pain : Ind EmotionalState ;   -- MidLevelOntology.gf:4402
```

where `EmotionalState` $\sqsubseteq$ `StateOfMind` $\sqsubseteq$ `PsychologicalAttribute` $\sqsubseteq \cdots \sqsubseteq$ `Attribute`. This makes `contraryAttribute(Pleasure, Pain)` type-correct by moving `Pain` from the Process branch to the Attribute branch—an *undocumented repair*.

This repair is defensible: in ordinary language, pain is a *sensation* (Attribute) that is *caused by* a pathological process, not the process itself. But the axiom usage census tells a different story: 5 axioms use `Pain` in Process-like contexts (`instance`, `located`, `WhenFn`) versus only 1 in an Attribute-like context (`contraryAttribute`). Raw evidence-counting would automate to the *wrong* classification.

The deeper judgment requires *causal analysis* (Pain is the effect, not the cause) and *similarity-class evidence* (`Pain` is used like `Pleasure`, which is `EmotionalState`, so Pain should be too). Our repair log (§B.5) records this decision with evidence, confidence, and canary theorems.

Behavioral dimension. Even if a permissive coercion `el_Process_Attribute` were added, $\Diamond$ would test whether it has non-trivial rewrite behavior. A vacuous coercion (no reduction rules) would be flagged as behaviorally empty—a diagnostic that the original SUMO-GF cannot provide.

Classmate arity mismatch (explained). SUMO declares `classmate` as a `TernaryPredicate` (arity 3: student, student, class) but also declares it an `IrreflexiveRelation`, whose axiom schema $\forall x.\ \neg R(x, x)$ assumes arity 2. In the SUMO-GF encoding, `classmate` has type:

$$\texttt{El_Agent} \to \texttt{El_Agent} \to \texttt{El_Class} \to \texttt{SumoFormula}$$

The binary irreflexivity pattern cannot match this ternary signature. The repair (from §9) is to replace the binary axiom with the properly quantified: $\forall a, c.\ \neg\texttt{classmate}(a, a, c)$.

B.5 Comparison with the Original sumo-gf

We obtained the original SUMO-GF source files from the GF 3.3 distribution (30 483 lines across 18 `.gf` files, including 1 722 `fun` declarations in `Merge.gf` and 2 827 in `MidLevelOntology.gf`; measured in our local copy of that distribution). Comparing the original with our replication reveals both *silent repairs* and *design choices* in the original encoding that were not discussed in the published work.

Silent repairs. The original SUMO-GF contains at least one undocumented reclassification (`Pain`: `PathologicProcess` → `EmotionalState`). Our pipeline, by faithfully following SUMO's own declarations, *detects* the conflict rather than silently resolving it. The repair loop (§6) then provides a principled framework for making and documenting such decisions.

Variable-arity relations. The original encoding correctly types `contraryAttribute` as `[El Attribute] -> Formula` (a GF list type), respecting SUMO's declaration as a `VariableArityRelation`. Our initial encoding simplified this to a binary signature—a design choice that should be corrected. This repair is fully automatable: any relation declared `(instance R VariableArityRelation)` should receive a list-type signature (a mechanically checkable transformation in this encoding pipeline).

Unresolved inheritance proofs. The axiom files (`.gft`) contain 592 lines with unresolved `?` metavariables (out of 1 184 axiom lines)—these represent coercion proofs that GF's type inference could not automatically resolve. The pattern `var ? ? ? VARIABLE` indicates that the class, target class, and inheritance proof for a variable coercion are all unknown. In our pipeline, these would be flagged as incomplete behavioral types requiring either explicit coercion paths or evidence-based inference.

Enache's actual statistics. Enache's thesis reports that 64% of quantified axioms passed type-checking [23]. The 36% failure breaks down as:

- 12% genuinely rejected by the type checker
- 12% inexpressible (variable types from usage context)
- 11% using untranslated predicates (`range`, `domain`)
- 1% other

For the higher-order axioms (involving meta-properties like `AsymmetricRelation`), only 43% translated successfully, largely because `AsymmetricRelation` in GF assumes same-type arguments while SUMO applies it to cross-type relations like `frequency : Process → TimeDuration` [23].

Evidence-based classification automation. Our analysis suggests a three-tier automation strategy for classification judgments:

1. **Axiom usage census**: Count how a concept is used across all SUMO axioms. Process-like patterns: `instance`, `located`, `WhenFn`, `agent`, `patient`. Attribute-like patterns: `attribute`, `contraryAttribute`, `property`. This is fully automatable.
2. **Similarity-class evidence**: If concept X is used in the same axiom patterns as concept Y, and Y has a known classification, then X should likely share that classification. For Pain: `Pain` appears alongside `Pleasure` in `contraryAttribute`, and `Pleasure` is `EmotionalState`, so Pain should be too. This is automatable via co-occurrence analysis.
3. **Causal/literature analysis**: When census and similarity evidence conflict (as for Pain: 5 Process-like uses vs. 1 Attribute-like use, but similarity says Attribute), deeper judgment is needed. An LLM or commonsense KB can distinguish "X is used as Y" from "X *is* Y" by identifying causal structure (Pain is *caused by* a process but *is* a sensation).

Each classification decision is recorded in a structured *repair log* (implemented as a Lean 4 structure with fields for original classification, evidence items, confidence, automation method, and canary theorems). This log serves as an audit trail that the original SUMO-GF lacks.

What the pipeline adds. Beyond reproducing Enache's sort-level detections, our pipeline provides:

1. **Rewrite dynamics**: The $\Diamond/\Box$ operators test whether coercions and axioms have non-trivial operational content, not just type-correctness.
2. **Proven Galois connection**: The adjunction $\Diamond \dashv \Box$ is proven automatically for the SUMO-GF language definition (via `langGalois`).
3. **Repair audit trail**: Every classification decision is logged with evidence, confidence, and canary theorems—making the encoding *reproducible and auditable*, unlike the original's silent repairs.
4. **Cross-grammar invariance**: The same logical form can be expressed in both the RGL (English) grammar and SUMO-GF. Agreement in OSLF type assignment provides a grammar-invariant correctness check.

B.6 Implementation

The encoding is implemented in five Lean 4 files:

- `SumoAbstract.lean`: 57 class definitions, 54 subclass edges, ∼350 function signatures (logical connectives, quantifiers per class, coercions per edge, SUMO relations)
- `SumoOSLFBridge.lean`: Rewrites, the `LanguageDef`, OSLF type system, Galois connection proof, and executable pipeline tests
- `SumoNTT.lean`: Class hierarchy as GSLT, NTT extraction, comprehensive audit (26 decisions, 20 testable)
- `EvidenceModel.lean`: Repair archetypes, evidence atoms, review voting, assurance scoring (282 lines)
- `RepairLog.lean`: 26 repair decisions with evidence, WM witnesses, and review state tracking

Count provenance (snapshot): generated from these modules plus `Mettapedia/Languages/GF/SUMO/audit_repair_` and `Mettapedia/Languages/GF/SUMO/data/pipeline_audit.json`. In the reported snapshot, the listed pipeline files build clean; this claim is scoped to those files rather than the entire repository. The Galois connection $\Diamond \dashv \Box$ for SUMO-GF is proven automatically by the same `langGalois` lemma used for English and Czech.

B.7 Class Hierarchy as GSLT: NTT Extraction

A key discovery is that the SUMO class hierarchy *is itself* a graded semantic lattice theory (GSLT). The subclass relation `Human` $\sqsubseteq$ `CognitiveAgent` $\sqsubseteq$ `SentientAgent` $\sqsubseteq \cdots \sqsubseteq$ `Entity` is a rewrite system: each coercion `el_`c_1`_`$c_2(x) \Rightarrow x$ models subsumption as reduction. With the lattice operations `both` (meet) and `either` (join) from GF's `Basic.gf`, this forms a GSLT.

We construct a fresh `LanguageDef` (`sumoHierarchyLangDef`) where:

- **Sorts** = SUMO class names (33 for Layer 1)
- **Terms** = lift constructors + coercion rules (from the transitive closure: 153 edges from 34 direct edges)
- **Rewrites** = subclass-as-reduction (`lift_`$c_1(x) \Rightarrow$ `lift_`$c_2(x)$ for each $c_1 \sqsubseteq c_2$)

Passing this through `langOSLF` automatically produces $\Diamond \dashv \Box$ and NTT. The Galois connection is proven by the same `langGalois` lemma.

Behavioral types. The *behavioral type* of a concept X is the set of sorts reachable via subclass coercion:

$$BT(X) = \{S \mid \Diamond(is_S)(X)\}$$

For example:

$$\begin{aligned} BT(\texttt{CognitiveAgent}) &= \{\texttt{CognitiveAgent}, \texttt{SentientAgent}, \texttt{AutonomousAgent}, \texttt{Object}, \texttt{Physical}, \texttt{Entity}\} \\ BT(\texttt{Process}) &= \{\texttt{Process}, \texttt{Physical}, \texttt{Entity}\} \\ BT(\texttt{Attribute}) &= \{\texttt{Attribute}, \texttt{Abstract}, \texttt{Entity}\} \end{aligned}$$

Automated gap detection. A *type gap* occurs when a concept X is used in an axiom context demanding sort S, but $S \notin BT(X)$. The NTT extraction on Layer 1 (strata 0–1) automatically detects four gaps:

Concept	Relation	Demanded sort	Fix
`Pain`	`contraryAttribute`	`Attribute`	split/reclassify
`VirtuousAgent`	`attribute`	`Attribute`	FOET modeling error
`(our sig)`	`capableInSituation` arg 2	`CaseRole`	encoding fixed
`(our sig)`	`capableInSituation` arg 3	`Object`	encoding fixed

Gaps 3–4 were encoding errors in our replication (wrong KIF domain readings); these are fully automatable and have been corrected. Gap 2 is a genuine modeling error in FOET: `VirtuousAgent` is a *class* of agents ($\sqsubseteq$ `AutonomousAgent`), not an `Attribute` value, yet FOET uses `(attribute ?AGENT VirtuousAgent)` at line 4733 of the seed ontology. The correct SUMO pattern is to create an individual attribute (cf. `(instance Pietas VirtueAttribute)` at the same file's line 4739, correctly used as `(attribute ?AGENT Pietas)`). Gap 1 (Pain) requires causal/literature evidence as discussed above.

Variable-name type inference. SUMO KIF variable names carry implicit type information: `?AGENT` implies `AutonomousAgent`, `?PROC` implies `Process`, `?ATTR` implies `Attribute`, and so on. While imprecise, these conventions enable *type-directed hole-filling*: given a type gap where concept X fills a slot demanding sort S, we compute the *inverse behavioral type* $BT^{-1}(S) = \{C \mid S \in BT(C)\}$—the set of all concepts that could type-correctly fill the slot. For example, the 7 inhabitants of `Attribute` in Layer 1 are: `Attribute`, `InternalAttribute`, `BiologicalAttribute`,

`PsychologicalAttribute`, `RelationalAttribute`, `NormativeAttribute`, `ObjectiveNorm`. `VirtuousAgent` is not among them, confirming the type gap.

Combined with pattern matching against analogous correct usage in the same file (e.g., `(instance Pietas VirtueAttribute)` followed by `(attribute ?AGENT Pietas)` at FOET lines 4739–4743), the auto-fix rule becomes:

> **IF** $S \notin BT(X)$ **AND** $\exists$ axiom using `(instance _ X)` **AND** $\exists$ nearby `(instance Y S')` with $S' \sqsubseteq S$ used as `(relation _ Y)`, **THEN** propose replacing `(relation _ X)` with `(instance _ X)`.

This rule detected the `VirtuousAgent` instance/class confusion in FOET and proposed the correct fix automatically.

AsymmetricRelation and vacuous axioms. Enache's GF encoding of `AsymmetricRelation` assumes same-type arguments: `(c:Class)` $\rightarrow$ `(El c` $\rightarrow$ `El c` $\rightarrow$ `Formula)` $\rightarrow$ `Formula`. This caused 57% of higher-order axioms to fail translation. However, examining the 13 `AsymmetricRelation` instances in `Merge.kif` reveals that 4/13 have domains on *disjoint branches* (e.g., `attribute : Object` $\times$ `Attribute`, `manner : Process` $\times$ `Attribute`). For cross-type relations, asymmetry is *vacuously true*: if $R : C_1 \times C_2$ with C_1 and C_2 on disjoint branches, then $R(y, x)$ is ill-typed when $y : C_2$, $x : C_1$—the type system prevents the symmetric case.

Thus Enache's "failure" to translate these axioms was *accidentally correct*: omitting a vacuously true axiom is sound. The proper resolution is:

- **Same-branch** (9/13): keep the axiom—asymmetry has real content (e.g., $\texttt{properPart}(x, y) \rightarrow \neg\texttt{properPart}(y, x)$).
- **Disjoint-branch** (4/13): omit the axiom; the NTT verifies theorem that disjoint branch membership implies asymmetry.

The check is fully automatable: read `(domain R 1 C1)` and `(domain R 2 C2)`; if $BT(C_1) \cap BT(C_2) = \{\texttt{Entity}\}$, asymmetry is vacuous.

Full FOET repair results. Applying the NTT pipeline to the FOET seed ontology (6 700+ lines) detected 20 repair items: Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/SumoNTT.lean`, `Mettapedia/Languages/GF/SUMO/RepairLog.lean`, `Mettapedia/Languages/GF/SUMO/data/decision_coverage`

#	Issue	Fixed?	Conf.	Auto?
1	Pain classification (cross-branch)	No	0.80	No
2	`contraryAttribute` arity	Partial	0.95	Yes
3	`attribute` domain (Agent→Object)	Yes	1.00	Yes
4	AutonomousAgent (14 relations demand it)	Yes	0.95	Yes
5	SentientAgent hierarchy	Yes	0.95	Yes
6	Relation hierarchy (infra sorts)	Yes	0.85	Yes
7–8	Transitive closure + domain repair	Yes	1.00	Yes
9	AsymmetricRelation (vacuous for 4/13, verified)	Yes	0.95	Yes
10	VirtuousAgent instance/class	Yes	0.90	Yes
11–12	`capableInSituation` arity + domains	Yes	1.00	Yes
13	`interferesWith` removed (fabricated)	Yes	1.00	Yes
14	Pietas args swapped (3✓ vs 1×)	Yes	0.98	Yes
15	Process in Object slot: `attribute`→`property`	Yes	0.92	Yes
16	`contraryAttribute` with classes→schema	Yes	0.95	Yes
17	Pain in FOET `attribute`	No	0.85	No
18	`attribute` args swapped (6361)	Yes	0.95	Yes
19	`holdsEthicalPhilosophy` swapped	Yes	0.90	Yes
20	Syntax errors (14 items batch)	Yes	1.00	Yes

In the current repair-log snapshot, 20 items are tracked; 18 are fully automatable. The two requiring judgment both involve Pain's cross-branch classification (Decisions 1 and 17). The dominant repair patterns are:

1. **Counting**: correct vs incorrect usages of the same concept resolve ambiguity (Decisions 14, 18, 19).
2. **Definitional unfolding**: if X has a well-typed definitional expansion, replace the ill-typed shorthand with the expansion (Decision 10: VirtuousAgent $\rightarrow \exists V.\ \texttt{VirtueAttribute}(V) \wedge \texttt{attribute}(A, V)$).
3. **Domain lookup**: read `(domain R N C)` from KIF and generate signatures mechanically (Decisions 3, 7, 11, 12).
4. **Vacuity detection**: cross-type higher-order axioms are vacuously true by type disjointness (Decision 9).
5. **Superrelation widening**: when $S \notin BT(X)$ for relation R, check if R has a superrelation R' via `(subrelation R R')` whose domain *does* cover X's sort. Decision 15: 7 FOET axioms used `(attribute ?ACT ?VIRTUE)` where `?ACT` is a Process. Since `(subrelation attribute property)` and `(domain property 1 Entity)`, the fix is `attribute`→`property`—preserving semantics (every `attribute` fact entails the corresponding `property` fact) while widening the domain to include Processes.

WM evidence evaluation. For candidates requiring semantic judgment, the repair pipeline connects to OSLF Stage 3 (world-model evidence). For each candidate φ_i, we construct the evidence query $qsem(\neg\varphi_i)$—"is there evidence against φ_i?"—and evaluate it against available knowledge (ontology axioms, encyclopedias, domain standards). If the world model produces a counterexample, φ_i is too strong. The *most general unifier* (weakest surviving candidate) wins.

Application: Pain classification (Decision 1). Four candidates were evaluated:

	Candidate	Refuted?	Key evidence
(a)	`PathologicProcess` (status quo)	Yes	IASP, SEP, SNOMED, SUMO's own axioms
(b)	`EmotionalState` (Enache)	No	But too narrow (misses sensory)
(c)	`StateOfMind`	No	IASP: "psychological state"; broadest viable
(d)	Split: Sensation + Process	No	Clean but adds complexity

Candidate (a) was refuted by 7 sources (IASP 2020 definition: "an unpleasant sensory and emotional *experience*"; Stanford SEP: pain as quale/mental state; SNOMED CT: "sensation quality"; SUMO's own documentation calling Pain "a physical *sensation*"; and SUMO's internal inconsistency where `Medicine.kif` uses `(attribute ?P Pain)`). Candidate (c) is the MGU: `StateOfMind` $\sqsubseteq$ `PsychologicalAttribute` $\sqsubseteq$ `Attribute`, making `contraryAttribute` type-correct while being broader than `EmotionalState` to capture pain's sensory component.

This resolves the last remaining judgment-dependent repair in the current snapshot: of 20 decisions in the repair log, **19 are now fully automatable**, with Pain resolved via WM evidence evaluation. Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/RepairLog.lean`, `Mettapedia/Languages/GF/SUMO/DocEvidence.lean`, `Mettapedia/Languages/GF/SUMO/stratum1_completion_` `Mettapedia/Languages/GF/SUMO/stratum1_with_foet_completion_certificate.txt`.

Significance. The full repair pipeline—NTT gap detection → candidate enumeration → WM evidence evaluation → MGU selection—is a closed loop connecting OSLF's three stages. Stage 1 (GF sort discipline) detects type gaps. Stage 2 (behavioral types via $\Diamond/\Box$) computes reachability. Stage 3 (world-model evidence) evaluates semantic candidates. The six repair patterns (counting, definitional unfolding, domain lookup, vacuity detection, superrelation widening, WM evidence evaluation) together resolve 95% (19/20) of the detected issues fully automatically in the current repair-log snapshot, with the remaining 5% (Pain) resolved semi-automatically via structured evidence gathering from domain standards (IASP), encyclopedias (Wikipedia, SEP), and clinical ontologies (SNOMED CT).

Formal inconsistency detection via the proven-sound checker. The full pipeline—SUMO KIF → GF Pattern → GSLT → OSLF → WM `checkLang`—formally detects inconsistencies using SUMO's own data against its own type constraints. For each axiom, we encode the arguments as `Pattern` trees positioned at their KIF-declared sort (e.g., `lift_Process(pain)` for Pain, `lift_Attribute(pleasure)` for Pleasure), then evaluate the formula $is_S \vee \Diamond(is_S)$ ("is on branch S or can reach branch S via coercion") using `checkLang` with the hierarchy GSLT as the `LanguageDef`.

For `(contraryAttribute Pleasure Pain)`, which requires both arguments to reach `Attribute`:

Argument	Starting sort	Reaches Attribute?	Result
Pleasure	`lift_Attribute`	`.sat`	Correct
Pain	`lift_Process`	`.unsat`	**Type gap**
Pain (repaired)	`lift_Attribute`	`.sat`	Fixed

The checker derives: arg_1 `.sat` $\wedge$ arg_2 `.unsat` = axiom is **ill-typed**—an inconsistency in SUMO's own type system, detected formally and proven sound via `checkLangUsing_sat_sound`. After reclassifying Pain to `EmotionalState` (Attribute branch), both arguments return `.sat` and the axiom becomes well-typed.

No external evidence is needed for *detection*—SUMO's own KIF data, run through the pipeline's proven-sound checker, produces the inconsistency verdict. External evidence (IASP, similarity-class counting) is needed only for choosing *which repair* to apply.

A comprehensive audit ran `checkLang` against all 26 repair decisions: 20/26 are testable as reachability queries (the remaining 6 are encoding choices, meta-decisions, or syntax fixes). All 20 testable decisions passed—confirming that each detected gap is a genuine type error and each repair resolves it. The audit itself caught a test setup error (stratum 2 concept tested against a Layer 1 GSLT), demonstrating the value of systematic verification. The expanded audit (D22–D26) adds stratum-based checks for LinguisticExpression, Artifact, CognitiveAgent, and Group, with bidirectional testing (pre-fix `.unsat` + post-fix `.sat`) for typed-argument-swap and domain-narrowing archetypes. Results are persisted as structured WM witnesses in the Evidence Model (§7). Artifact pointers (snapshot): `Mettapedia/Languages/GF/SUMO/SumoNTT.lean`, `Mettapedia/Languages/GF/SUMO/data/wm_witnesses.jsonl`, `Mettapedia/Languages/GF/SUMO/data/decision`

Related work. This approach complements existing ontology debugging work: Álvez et al. (2019) developed white-box test generation for FOL ontologies (applied to Adimen-SUMO); Kalyanpur et al. (2005) introduced MUPS-based root cause analysis. Our contribution is the connection to OSLF's behavioral type theory: the class hierarchy as GSLT provides a *semantic* foundation for gap detection (not just syntactic test generation), and the WM evidence calculus provides a principled framework for candidate evaluation (not just user interaction). The proven-sound checker provides a formal guarantee that "`.sat` ⇒ `sem` holds" (Theorem `checkLangUsing_sat_sound`) in this pipeline slice; in the cited ontology-debugging tooling above, we are not aware of an equivalent theorem-level checker soundness result.

```
Entity
+-- Physical
|   +-- Object
|   |   +-- SelfConnectedObject
|   |   |   `-- CorpuscularObject
|   |   |       `-- ContentBearingObject
|   |   `-- AutonomousAgent <- VirtuousAgent, ViciousAgent
|   |       `-- SentientAgent
|   |           `-- CognitiveAgent
|   +-- Process <- AutonomousAgentProcess
|   |   +-- VirtuousAct, ViciousAct
|   |   +-- BodyMotion, Vocalizing
|   |   `-- Change
|   |   `-- LinguisticCommunication
|   |       `-- Conjecturing
|   +-- ContentBearingPhysical
|   |   `-- LinguisticExpression
|   |       `-- Sentence
|   |           `-- Formula <- ActionFormula, Conjecture
|   `-- Situation [FOET-defined]
|
`-- Abstract
    +-- Attribute
    |   +-- InternalAttribute
    |   |   `-- BiologicalAttribute
    |   |       `-- PsychologicalAttribute
    |   |           +-- VirtueAttribute [FOET]
    |   |           `-- ViceAttribute [FOET]
    |   `-- RelationalAttribute
    |       `-- NormativeAttribute
    |           +-- ObjectiveNorm
    |           |   `-- DeonticAttribute [FOET]
    |           `-- MoralAttribute [FOET]
    |               +-- MoralValueAttribute [FOET]
    |               `-- MoralVirtueAttribute [FOET]
    +-- SetOrClass
    |   +-- Class
    |   `-- Set <- ChoicePoint
    +-- Relation
    |   +-- Predicate (BinaryPredicate, TernaryPredicate, ...)
    |   +-- Function (UnaryFunction, BinaryFunction)
    |   |   `-- UtilityFormulaFn [FOET]
    |   `-- InheritableRelation
    |       `-- PropositionalAttitude (desires, conjectures)
    +-- Proposition
    |   `-- Argument <- ValidDeductiveArgument
    |       `-- ConsequentialistArgument [FOET]
    +-- List <- SentenceList [FOET]
    `-- Value [FOET] <- 10 Schwartz values
```

Figure 1: SUMO class hierarchy from `Entity` to FOET leaf concepts. Nodes marked [FOET] are introduced by the ethics ontology; all others are SUMO core (Merge.kif). Every path from root to leaf must be fully debugged.